Signals and Systems Project

Phase 2

Amirreza Mirjalily - 402102549

Sharif University of Technology

Analysis of Phase-Amplitude Coupling during Olfactory Stimulation as a Biomarker for Alzheimer's Disease in EEG Signals

---

Note that this report has been done using LaTeX.

Instructor: Prof. Hamid Aghajan

Sharif University of Technology,
Electrical Engineering Faculty

Summer 2025

Table of Contents

1 Preface . . . 2

2 Preparing the Datasets . . . 3

2.1 Importing the Necessary Libraries . . . 3

2.2 Loading the Datasets . . . 3

3 Filter Implementation . . . 4

4 MVL PAC Computation (Approach #1) . . . 5

4.1 Extracting the Proper Phase and Amplitude . . . 5

4.2 Computing PAC Using Sliding Window . . . 5

4.3 Averaging and Plotting . . . 7

4.3.1 Key Observations . . . 8

4.4 PAC Values for Channels . . . 9

4.4.1 Key Observations . . . 12

4.5 Specific Electrodes Selection . . . 13

4.5.1 Key Observations . . . 15

4.6 Repetition Effect . . . 15

4.6.1 Key Observations . . . 17

4.7 Effect of Stimulation Type . . . 18

4.8 Key Observations . . . 21

5 MVL PAC Computation (Approach #2) . . . 22

5.1 Extracting the Proper Phase and Amplitude . . . 22

5.2 Computing PAC Using Sliding Window . . . 22

5.3 Saving the Data . . . 23

6 MI PAC Computation . . . 25

6.1 P Distribution Calculation . . . 25

6.2 MI Calculation . . . 27

6.3 Averaging Across Epochs and Channels . . . 28

6.3.1 Key Observations . . . 28

6.4 Specific Electrode Selection . . . 28

6.4.1 Key Observations . . . 29

6.5 Phase-Amplitude Histogram . . . 29

6.5.1 Key Observations . . . 31

7 Comparison: MVL vs. MI . . . 32

1 Preface

Because of my recent switch to the Linux operating system and MatLab not being available on Linux for free, and my familiarity with the Python programming language, I decided to do the simulations and analysis regarding this phase of the project in **Python**, mainly using the **NumPy** library and **Matplotlib** for plotting.

There will be a **Jupyter Notebook** attached to this document containing all of the Python code and analysis results of this phase of the project.

There will also be 3 `.mat` files attached to this document that contain the PAC data for all of the 3 patients using the MVL method. These files contain both the PAC data and the center of the windows used for PAC calculation with the labels `PAC_final` and `time_pac` in order.

Because of my lacking knowledge in this field, I tried my best to fact-check any analysis or observation that I state throughout this document.

2 Preparing the Datasets

After exporting the datasets for **Healthy Control, MCI and Mild** from MatLab (where we did the exploratory data analysis, pre-processing and power analysis in the previous phase of the project) in the `.set` file format, we load these datasets into Python using the `mne` library for handling EEG data in Python.

2.1 Importing the Necessary Libraries

```
import mne
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import hilbert
from scipy.io import savemat
```

2.2 Loading the Datasets

```
healthy = mne.io.read_epochs_eeglab("Datasets/Final_Healthy.set")
MCI = mne.io.read_epochs_eeglab("Datasets/FInal_MCI.set")
mild = mne.io.read_epochs_eeglab("Datasets/Final_Mild.set")

healthy_data = healthy.get_data()
MCI_data = MCI.get_data()
mild_data = mild.get_data()

healthy_data *= 1000000
MCI_data *= 1000000
mild_data *= 1000000
```

Here we export the raw EEG signals in the form of a NumPy array with the shape of (epochs, channels, samples).

Also all of the signals have been multiplied by a factor of 10^6 to convert the amplitudes from Volts to Microvolts for a better insight.

3 Filter Implementation

In this part, we implemented a band pass filter in the form of a Python function.

But first, we extract the sampling frequencies for all of the datasets and assert them to be equal.

I also used the FIR band-pass filter from the `mne` library for it to be EEG specific and me not having to worry about padding and other stuff. It has EEG signal specific default parameters that suit EEG signals much better than other band-pass filter like the one from the `SciPy` library.

```
theta = [(4, 6), (6, 8)]
gama = [(30, 35), (35, 40), (40, 45), (45, 50)]

pure_theta = (4, 8)
pure_gama = (30, 50)

Fs_Healthy = healthy.info['sfreq']
Fs_MCI = MCI.info['sfreq']
Fs_Mild = mild.info['sfreq']

assert Fs_Healthy == Fs_MCI == Fs_Mild
Fs = Fs_Healthy

def bandpass_filter(signal, Fs, fstart, fend):
    return mne.filter_data(signal, Fs, fstart, fend, method='fir', verbose=False)
```

For the band pass filter implementation, I used an FIR filter so the output returns zero after a finite duration.

4 MVL PAC Computation (Approach #1)

In this approach, the gamma and theta frequency band is not divided into multiple bands and the averaged over.
In fact, the entire band for both the gamma and theta band is used.

4.1 Extracting the Proper Phase and Amplitude

In this part, by filtering the data using the already implemented band-pass filter and extracting the phase and amplitude, we build the fundamentals for the PAC computation using the MVL method.

My choice of method for Phase-Amplitude computation was the Hilbert Transform because of its efficient algorithm and high temporal precision.

```
def extract_phase_amplitude(epoch, Fs):
    amplitude_signal = bandpass_filter(epoch, Fs, pure_gama[0], pure_gama[1])
    amplitude = np.abs(hilbert(amplitude_signal, axis = -1))

    phase_signal = bandpass_filter(epoch, Fs, pure_theta[0], pure_theta[1])
    phase = np.angle(hilbert(phase_signal, axis = -1))

    return phase, amplitude
```

It's also good to note that the PAC will be computed for each epoch and channel separately and averaged over epochs and channels.

4.2 Computing PAC Using Sliding Window

Here we implement the functions to compute $z(t)$ and MVL itself.

$$z(t) = A_\gamma(t).e^{j\phi_\theta(t)}$$

$$MVL = \left| \frac{1}{N} \sum_{t=1}^{N} z(t) \right|$$

```
def compute_z(amplitude, phase):
    return amplitude * np.exp(1j * phase)

def compute_MVL(z):
    return np.abs(np.mean(z))
```

Then we implement the function to calculate PAC over a window using windows with the duration of $100ms$ and 95% overlap. Then we move the window using a simple `while` loop and calculate PAC for each window and store them inside a dynamic array.

We use the sliding window technique to be able to be able to track the PAC value over time with a high enough accuracy.

```
def sliding_window_mvl(epoch, Fs):
    window_samples = int(window_duration * Fs)
    window_step = int((1 - window_overlap) * window_samples)
    start = 0
    signal_end = len(epoch)

    mvl = []
    phase, amplitude = extract_phase_amplitude(epoch, Fs)

    while True:
        if start >= signal_end-1:
            break

        if start + window_samples >= signal_end:
            end = signal_end - 1

        else:
            end = start + window_samples

        mvl.append(compute_MVL(compute_z(amplitude[start:end], phase[start:end])))

        start += window_step

    return np.array(mvl)
```

As I stated before, we will be calculating the PAC for all of the channels of all the epochs separately, so we define another function to take care of that and wrap all of the processes needed inside a single function.

```
def get_mvls(process_name, signal, Fs, epoch_count, channel_count, length):
    arr = np.zeros(shape = (epoch_count, channel_count, length))
    count = 0
    all_count = epoch_count * channel_count

    for i in range(epoch_count):
        for j in range(channel_count):
            arr[i, j, :] = sliding_window_mvl(signal[i, j], Fs, epoch_count, channel_cou
            count += 1
            progress_bar(process_name, count, all_count)

    return arr
```

In this function, we iterate over all of the epochs and channels and compute PAC using the sliding window method that has already been implemented. Then we store the results in an array with the shape of (epochs, channels, windows).

4.3 Averaging and Plotting

To be able to visualize PAC values over time on a single curve, we have to average the computed PAC values across all of the epochs and channels.

```
healthy_average = healthy_MVLs.mean(axis = 0)
MCI_average = MCI_MVLs.mean(axis = 0)
mild_average = mild_MVLs.mean(axis = 0)

healthy_MVL_final = healthy_average.mean(axis = 0)
MCI_MVL_final = MCI_average.mean(axis = 0)
mild_MVL_final = mild_average.mean(axis = 0)
```

After averaging, we can plot the PAC values and track them over time on a single curve.

```
time_axis = []
number_of_windows = len(healthy_MVL_final)
start = 0
step = int(0.05 * Fs)
window_size = Fs

for i in range(number_of_windows):
    time_axis.append(((start+(window_size/2))/Fs) * 1000)
    start += step

plt.figure(figsize = (10, 5))
plt.plot(time_axis, healthy_MVL_final, color='blue', label='Healthy')
plt.plot(time_axis, MCI_MVL_final, color = 'red', label = 'MCI')
plt.plot(time_axis, mild_MVL_final, color = 'green', label = 'Mild')
plt.xlabel('Time (ms)')
plt.ylabel('PAC')
plt.grid(True)
plt.legend()
plt.show()
```

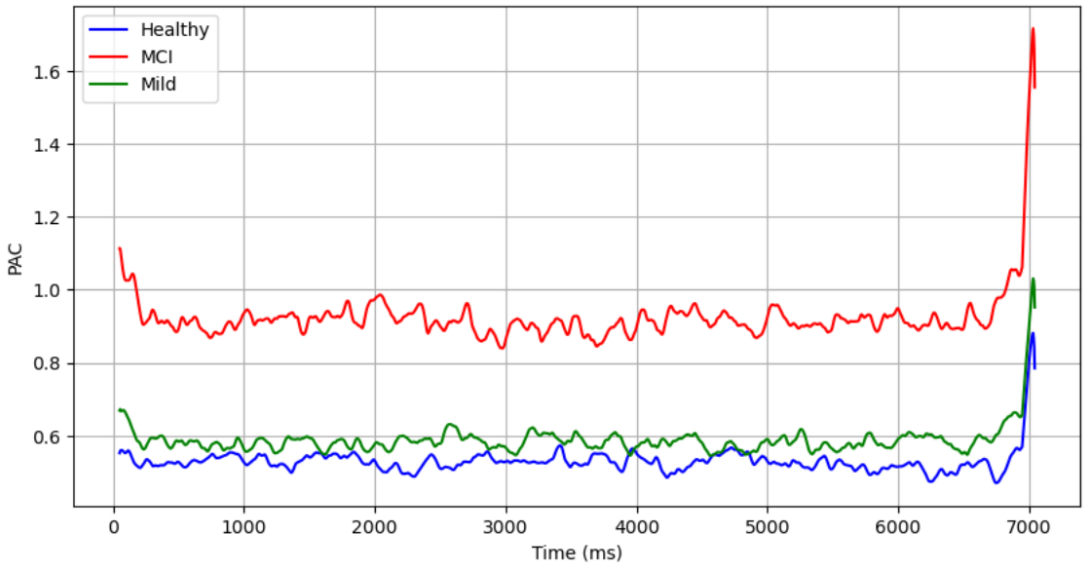

Figure 1: PAC Values over time for Healthy, MCI and Mild patients.

4.3.1 Key Observations

Because of my lacking knowledge in this field, my observations and interpretations might be flawed and scientifically wrong, but I tried my best to fact-check everything using online sources.

Even though we expected the PAC values to be higher for the healthy patient, we are clearly getting higher PAC values for the MCI and Mild patients in comparison with the Healthy patient.

According to some of the online sources, this might indicate more "struggle" or more effort being put in by the patient's brain to process a stimulus or a task.

I also double-checked the code and pre-processing to make sure that the analysis results are correct.

Also, the rising PAC values for all of the patients at the beginning and end of an epoch (relatively) indicate the fact that the PAC value increases when a new stimulus is activated.

This rise seems to be much more for the MCI and Mild patients in comparison with the healthy patient, which might indicate more struggle to process the stimulus.

4.4 PAC Values for Channels

In this part, the PAC values averaged across epochs are plotted for all of the channels. To avoid confusion, the channels are separated into 2 parts and plotted separately.

```
time_axis = []
number_of_windows = len(healthy_MVL_final)
start = 0
window_size = Fs * window_duration
step = int((1 - window_overlap) * window_size)

for i in range(number_of_windows):
    time_axis.append(((start+(window_size/2))/Fs) * 1000)
    start += step

plt.figure(figsize = (15, 8))

for i in range(int(healthy_channels/2)):
    plt.plot(time_axis, healthy_average[i], label = f'Channel {healthy.ch_names[i]}')

plt.xlabel('Time (ms)')
plt.ylabel('PAC')
plt.title('PAC per Epoch Plot for Healthy (Channels FP1 to P4)')
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize = (15, 8))

for i in range(int(healthy_channels/2), healthy_channels):
    plt.plot(time_axis, healthy_average[i], label = f'Channel {healthy.ch_names[i]}')

plt.xlabel('Time (ms)')
plt.ylabel('PAC')
plt.title('PAC per Epoch Plot for Healthy (Channels T6 to F3)')
plt.grid(True)
plt.legend()
plt.show()
```

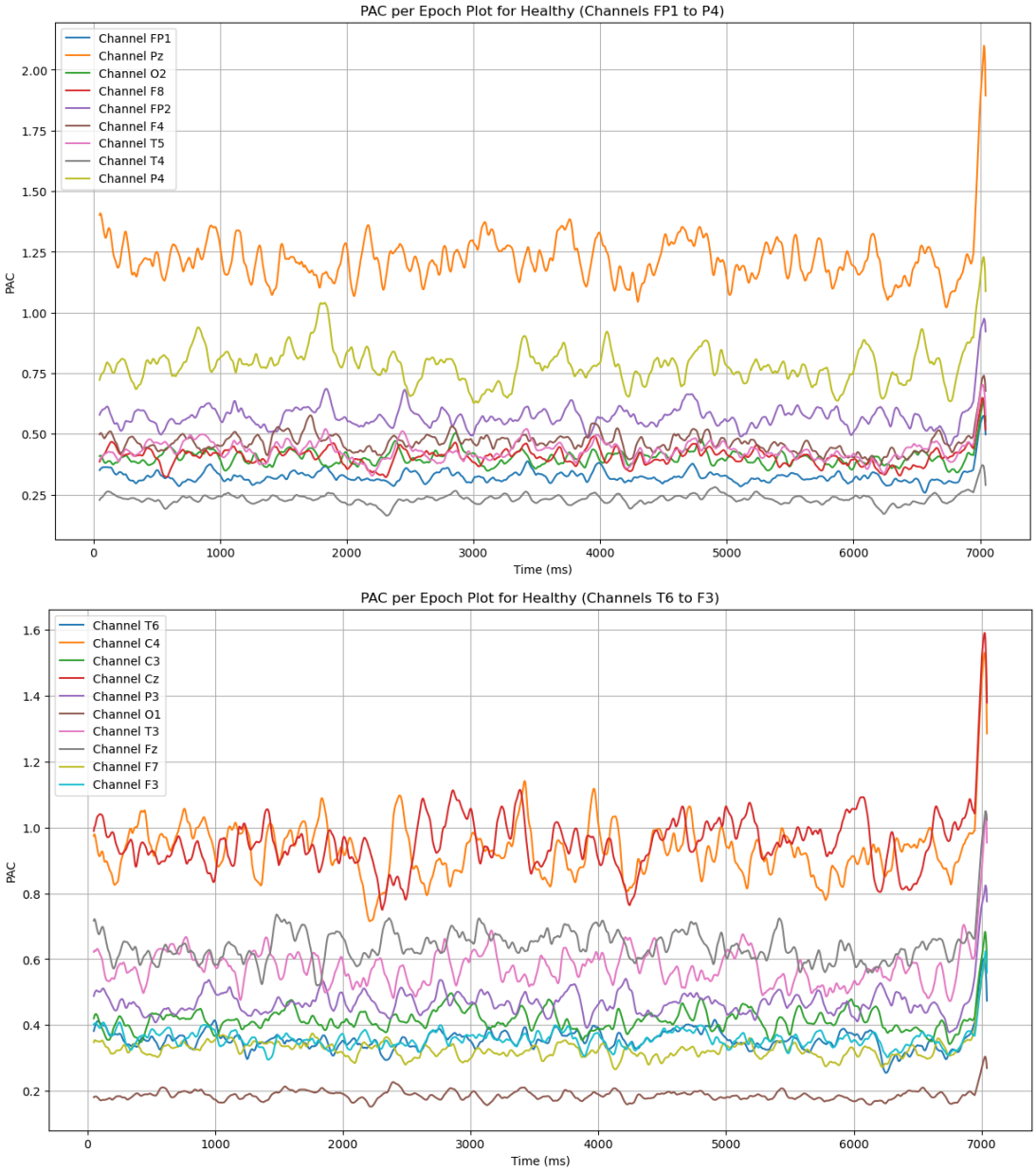

Figure 2: Healthy Patient

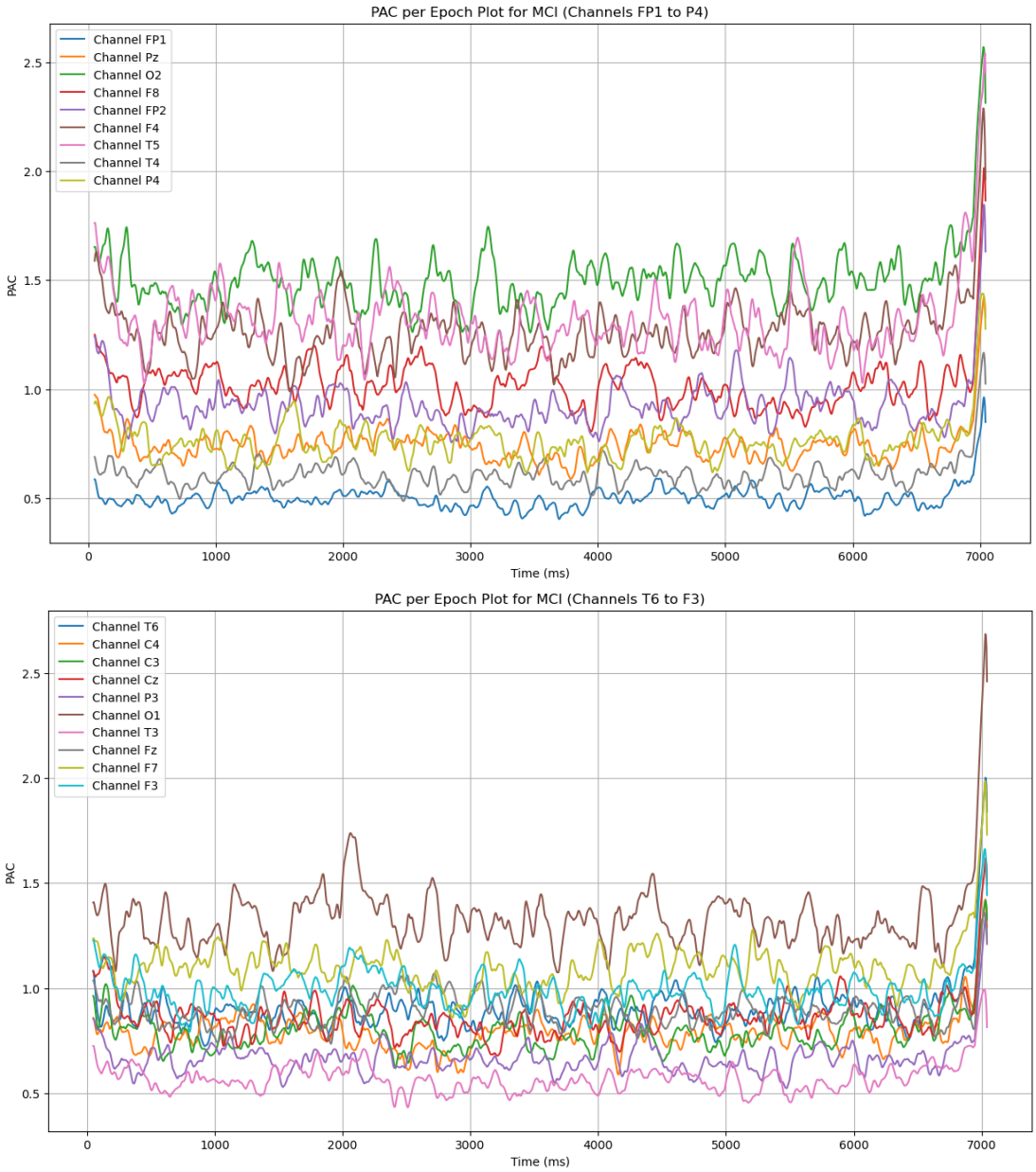

Figure 3: MCI Patient

Figure 4: Mild Patient

4.4.1 Key Observations

It's obvious that the PAC values are different from channel to channel for all of the patients. Some of the channels have higher or more stable PAC values within an epoch.

It also seems like the channels with a higher PAC value for the healthy patient have a much more stable PAC over an epoch in comparison with the channels with a higher PAC value for the 2 other patients.

Also in general, the PAC values within an epoch seem to be more stable for the healthy patient in comparison with the 2 others. (Averaging across channels made MCI and Mild patients seem to have stable PAC values, which is not true.)

4.5 Specific Electrodes Selection

In this part, we only select some of the electrodes (channels) to plot and observe the differences between different patients.

The selected electrodes are FP_1, FP_2, F_z and P_z.

```
selected_channels = (0, 4, 16, 1)

time_axis = []
number_of_windows = len(healthy_MVL_final)
start = 0
window_size = Fs * window_duration
step = int((1 - window_overlap) * window_size)

for i in range(number_of_windows):
    time_axis.append(((start+(window_size/2))/Fs) * 1000)
    start += step

plt.figure(figsize = (10, 5))

for i in selected_channels:
    plt.plot(time_axis, healthy_average[i], label = f'Channel {healthy.ch_names[i]}')

plt.xlabel('Time (ms)')
plt.ylabel('PAC')
plt.title('PAC per Epoch Plot for Healthy')
plt.grid(True)
plt.legend()
plt.show()
```

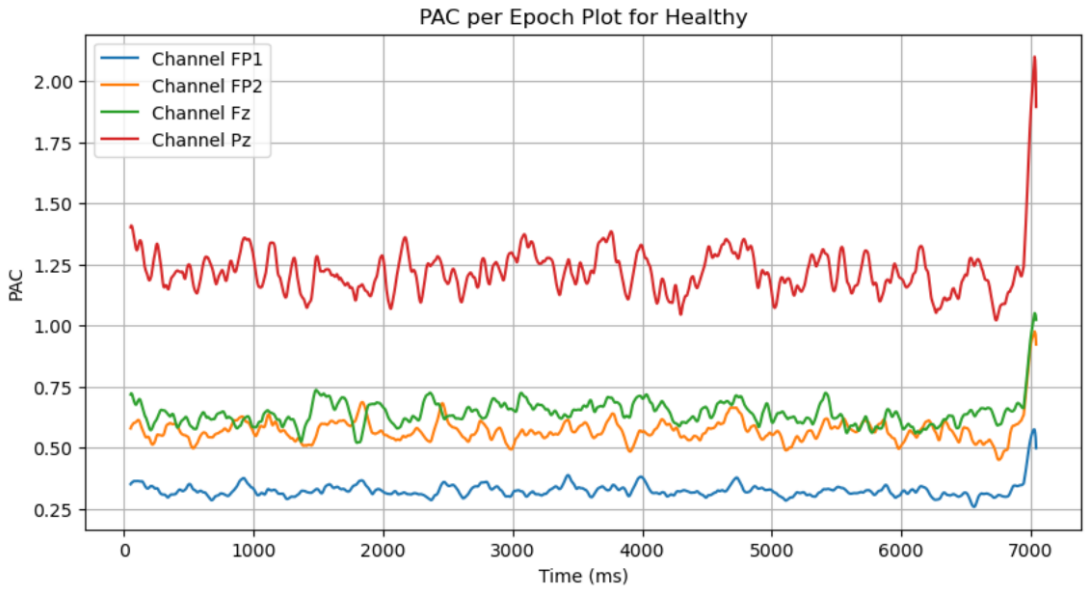

Figure 5: Healthy Patient

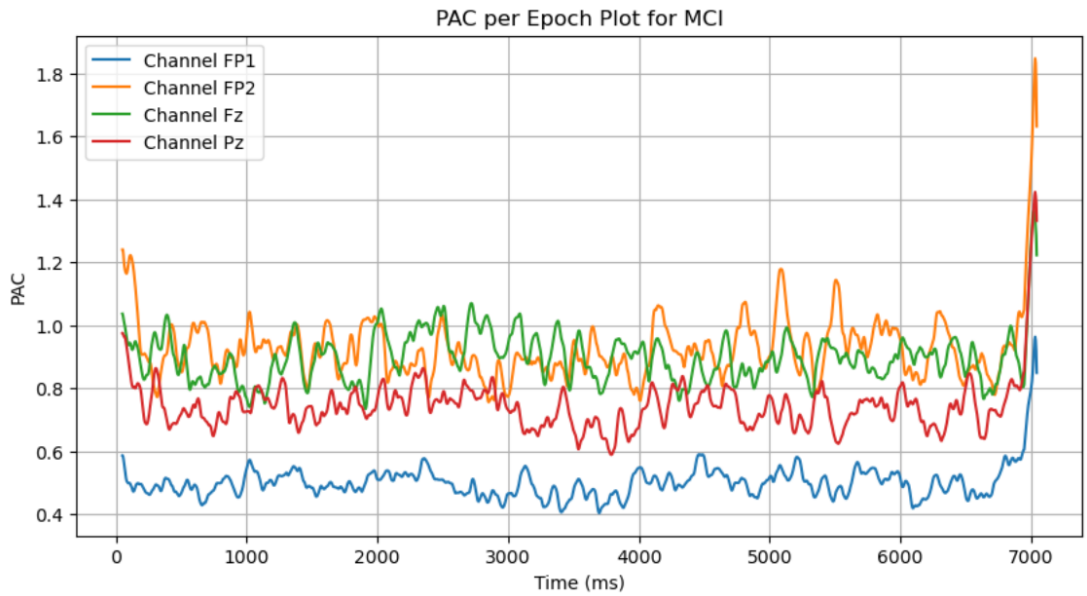

Figure 6: MCI Patient

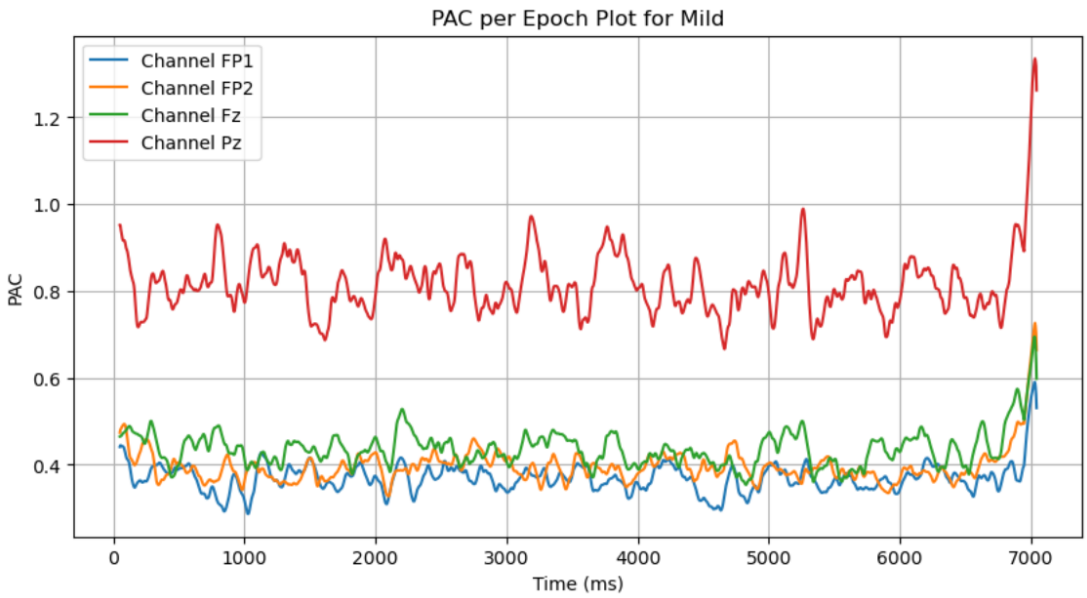

Figure 7: Mild Patient

4.5.1 Key Observations

It seems like the PAC values for these specific electrodes are much more steady and stable for the Healthy patient and do not tolerate that much. On the other hand, these values seem to be too noisy and tolerate a lot for the MCI and Mild patients.

Also, the Healthy patient has the highest PAC value on the P_z channel among all 3 of the patients (which is related to sight and visual attention).

4.6 Repetition Effect

The results of this part have been provided as an insight for bonus marks related to the *Additional Insights/Visualizations* part.

To be able to observe the effect of repetition on PAC values for all 3 of the patients, we average the PAC values over time and across all of the channels and plot them for all of the epochs.

```
healthy_epochs_data = healthy_MVLs.mean(axis = 1).mean(axis = 1)
MCI_epochs_data = MCI_MVLs.mean(axis = 1).mean(axis = 1)
mild_epochs_data = mild_MVLs.mean(axis = 1).mean(axis = 1)

healthy_epoch_enumerate = np.linspace(1, healthy_epochs, healthy_epochs)
MCI_epoch_enumerate = np.linspace(1, MCI_epochs, MCI_epochs)
mild_epoch_enumerate = np.linspace(1, mild_epochs, mild_epochs)

plt.plot(healthy_epoch_enumerate, healthy_epochs_data)
plt.title('Healthy Average across Channels')
```

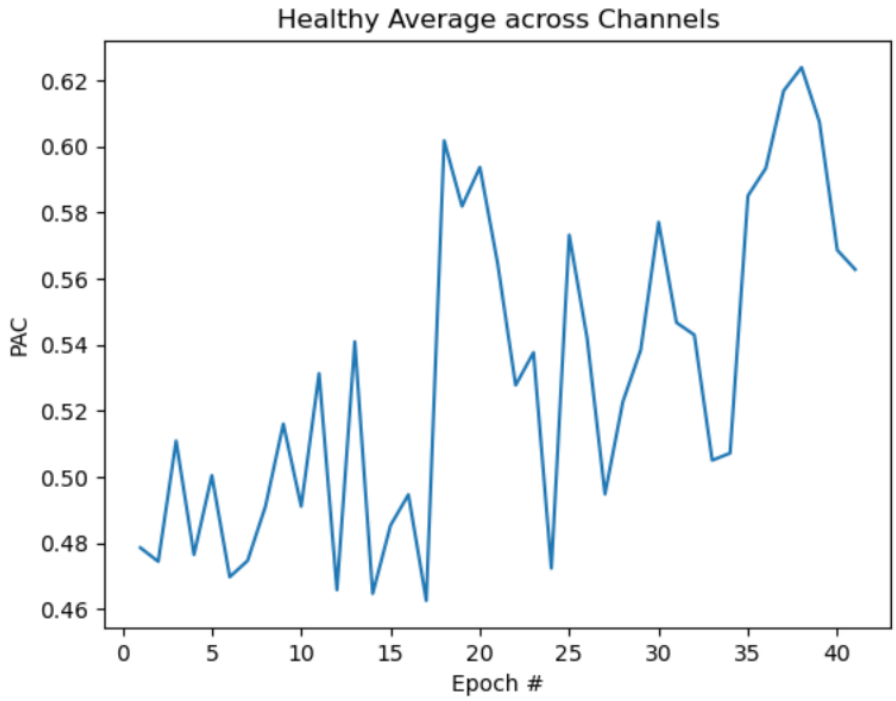

Figure 8: Effect of Repetition on the Healthy patient

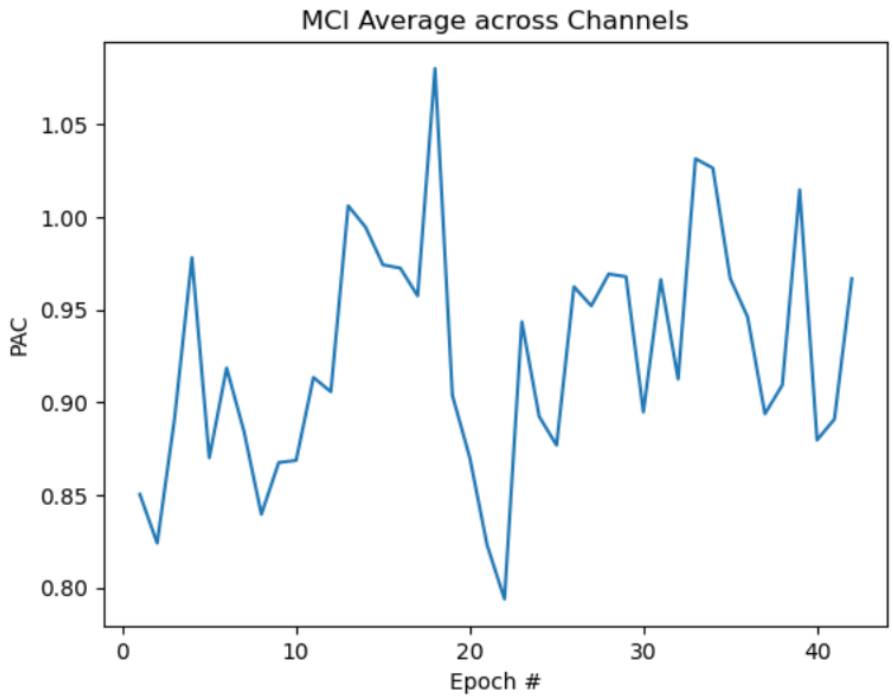

Figure 9: Effect of Repetition on the MCI patient

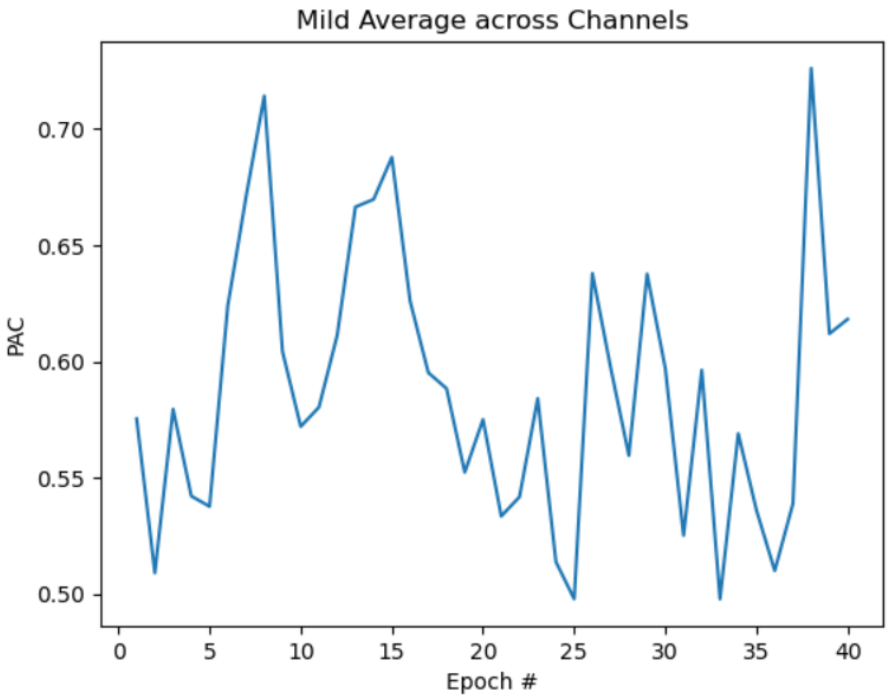

Figure 10: Effect of Repetition on the Mild patient

4.6.1 Key Observations

The plot for the Healthy patient seems to have an upward trend, which is expected because of the effect of repetition.

Also, the hills seem to be more stable for the Healthy patient and sharper for the MCI and Mild patients, which can be an indicator of lower short-term memory capabilities. (There is only one sharp hill seen on the Healthy plot.)

4.7 Effect of Stimulation Type

In this part, we would like to observe the effect of the kind of stimulation on the calculated PAC values.

For this matter, the PAC values are plotted across a epoch (averaged across epochs) for both the **Rose** and **Chocolate** stimulation individually.

```
rose_epochs = []
chocolate_epochs = []

rose = healthy.event_id['Rose']
chocolate = healthy.event_id['Chocolate']

index = 0

for event in healthy.events:
    if event[2] == rose:
        rose_epochs.append(index)

    else:
        chocolate_epochs.append(index)

    index +=1

plt.figure(figsize = (10, 5))
plt.plot(time_axis, healthy_MVLs[rose_epochs, :, :].mean(axis = (0, 1)), color='blue', l
plt.xlabel('Time (ms)')
plt.ylabel('PAC')
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize = (10, 5))
plt.plot(time_axis, healthy_MVLs[chocolate_epochs, :, :].mean(axis = (0, 1)), color='blu
plt.xlabel('Time (ms)')
plt.ylabel('PAC')
plt.grid(True)
plt.legend()
plt.show()
```

In this code, the indexes for Rose and Chocolate stimulation are separated and stored inside a dynamic array.

Then, these indexes are used to plot the PAC values over each epoch for both the Rose and Chocolate stimulation individually.

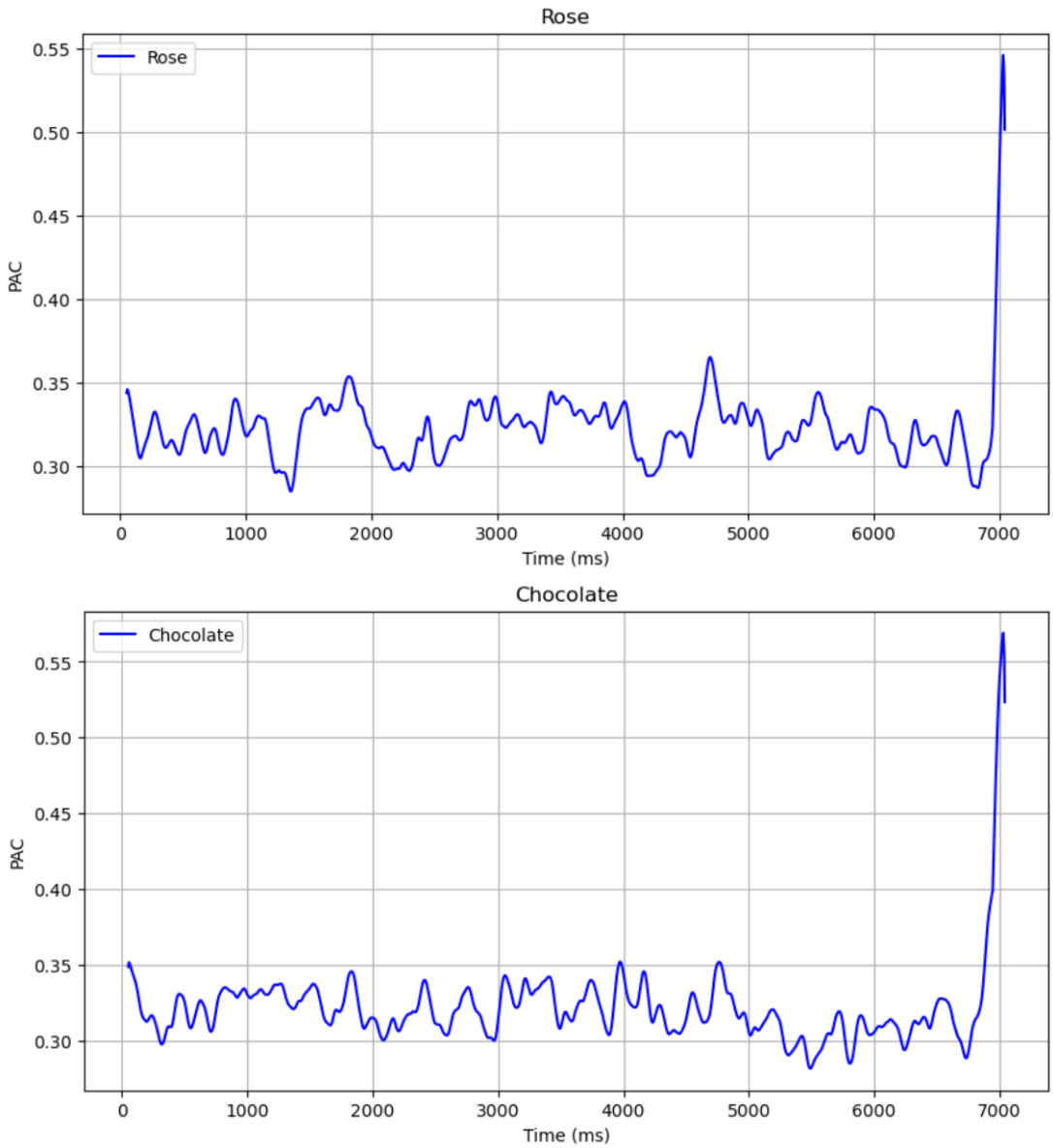

Figure 11: Rose and Chocolate PAC values for Healthy Patient

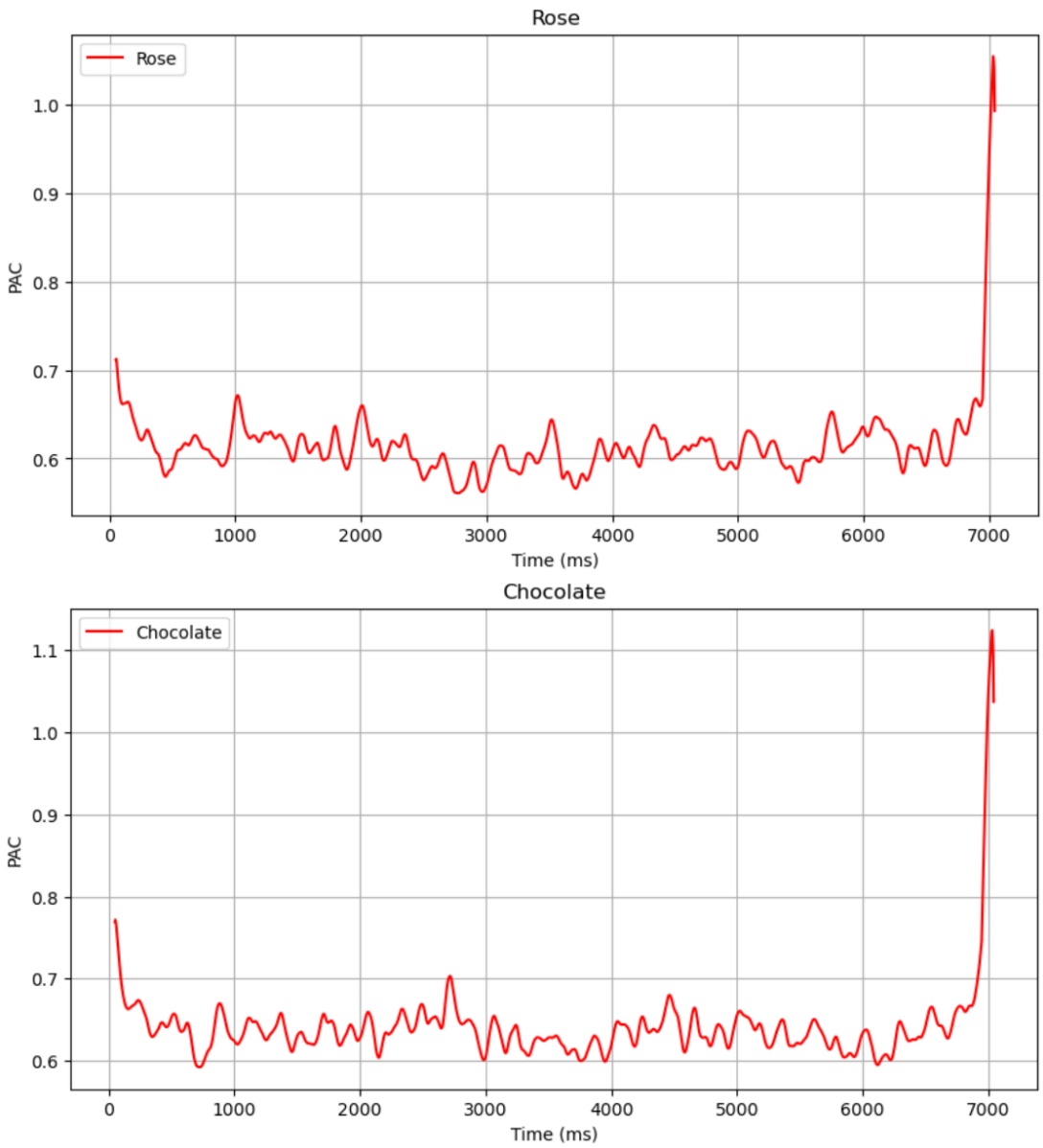

Figure 12: Rose and Chocolate PAC values for MCI Patient

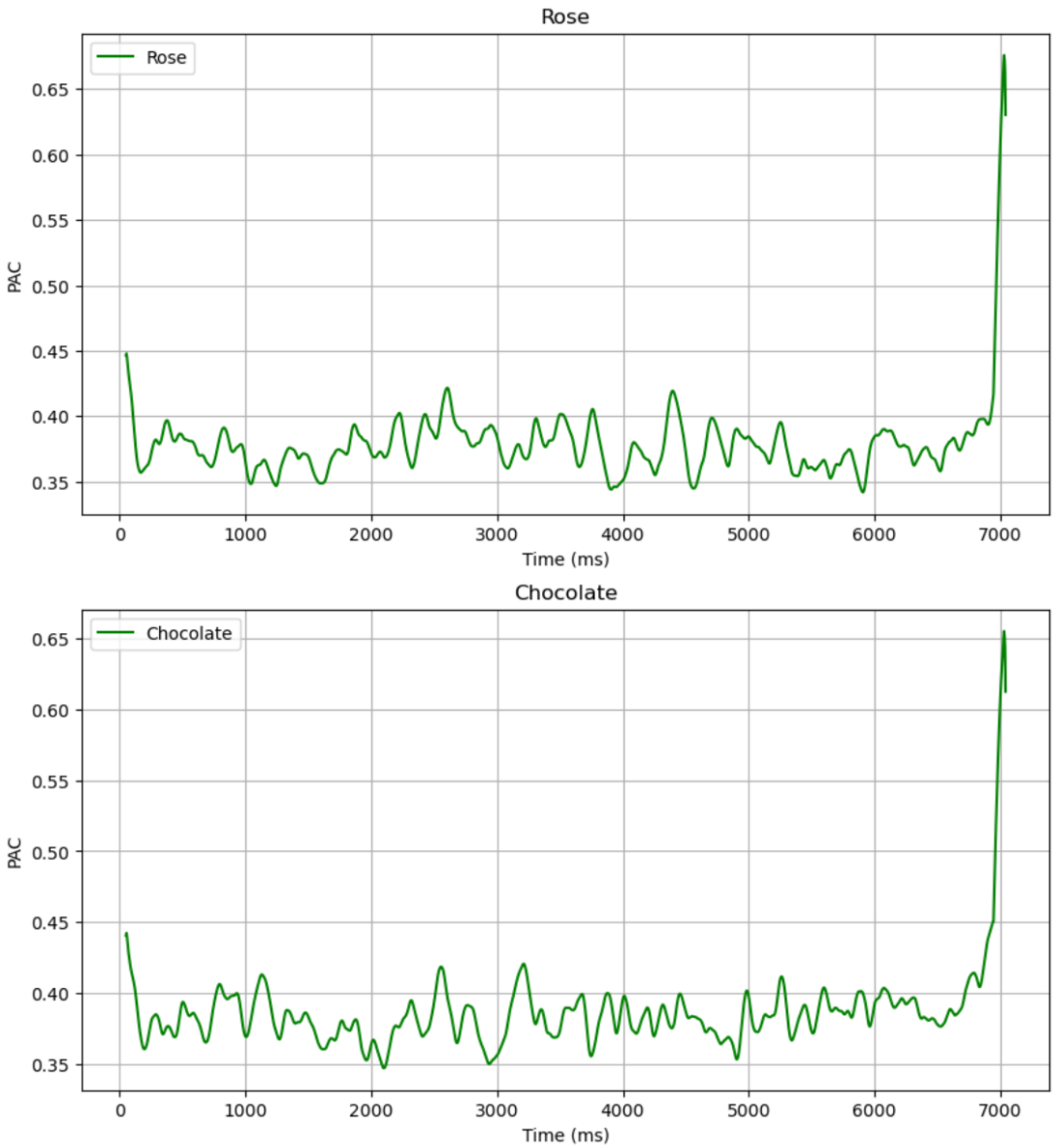

Figure 13: Rose and Chocolate PAC values for Mild Patient

4.8 Key Observations

It seems like the type of the applied stimulus does not affect the theta band phase and the gamma band amplitude coupling strength.

This means that our previous results are not biased by the type of the applied stimulus. (Because we averaged over all of the epochs regardless of their type.)

5 MVL PAC Computation (Approach #2)

In this approach, instead of using the entire gamma and theta bands directly, they are each divided into smaller bands, and the PAC is calculated using the MVL method for each of the 2 pairs possible.
This approach provides much more accuracy.

Note that the analysis results for the second approach do not provide any additional insights are not included in this document, how ever, they are available in the attached Jupyter Notebook.

5.1 Extracting the Proper Phase and Amplitude

This part is so similar to the corresponding part in the first approach.

The only major difference is that instead of having a single phase and amplitude, we have multiple phase and amplitude pairs because we divided the frequency band into multiple ones.

```
def extract_phase_amplitude(epoch, Fs):
    phases = []
    amplitudes = []

    for band in gama:
        amplitude_signal = bandpass_filter(epoch, Fs, band[0], band[1])
        amplitude = np.abs(hilbert(amplitude_signal, axis = -1))
        amplitudes.append(amplitude)

    for band in theta:
        phase_signal = bandpass_filter(epoch, Fs, band[0], band[1])
        phase = np.angle(hilbert(phase_signal, axis = -1))
        phases.append(phase)

    return np.array(phases), np.array(amplitudes)
```

5.2 Computing PAC Using Sliding Window

This part is also so similar to the corresponding part in the first approach.

The only major difference is that the PAC value is calculated for each of the frequency band pair and then averaged over all of the pairs for a higher accuracy.

```
def sliding_window_mvl(epoch, Fs):
    window_samples = int(window_duration * Fs)
    window_step = int((1 - window_overlap) * window_samples)
    start = 0
    signal_end = len(epoch)
```

```
    mvl = []
    phases, amplitudes = extract_phase_amplitude(epoch, Fs)

    for phase in phases:
        for amplitude in amplitudes:
            local = []

            while True:
                if start >= signal_end-1:
                    break

                if start + window_samples >= signal_end:
                    end = signal_end - 1
                else:
                    end = start + window_samples

                local.append(compute_MVL(compute_z(amplitude[start:end], phase[start:end

                start += window_step

            local = np.array(local)
            mvl.append(local)
            start = 0

    return np.array(mvl).mean(axis = 0)
```

5.3 Saving the Data

Since the PAC computation approach used in the second part is more accurate, these results are save instead of the first parts.

We save the PAC data calculated using the MVL method in a `.mat` file to be easily read by MatLab.

These files contain both the PAC data and the center of the windows used for the calculations having the labels `PAC_final` and `time_pac` in order.

```
savemat('PAC Data/Healthy_PAC.mat', {
    'PAC_final' : healthy_MVLs,
    'time_pac' : np.array(time_axis)
})

savemat('PAC Data/MCI_PAC.mat', {
    'PAC_final' : MCI_MVLs,
    'time_pac' : np.array(time_axis)
})

savemat('PAC Data/Mild_PAC.mat', {
```

```
    'PAC_final' : mild_MVLs,
    'time_pac' : np.array(time_axis)
})
```

6 MI PAC Computation

In this part, we calculate PAC for all of the 3 patients using the **MI method**.

Using the trial and error method, I chose my bin count to be 18, which is both not too high to be unable to give any insight and not too low to be inaccurate.

6.1 P Distribution Calculation

The first step in calculating the MI is finding the P distribution.

$$P(\theta_i) = \frac{A_i}{\sum_{j=1}^{n} A_j}$$

We extract the signal's amplitude in the gamma band and its phase in the theta band. Then we divide the phase into N equal bins from $-\pi$ to π.

This way, A_i corresponds to the average gamma amplitude that falls within that particular phase bin.

After that, we normalize this value by dividing it by the sum of all A_is, and obtain a probability distribution.

It's also good to note that the MI value will be calculated for each epoch and each of its channels individually and then averages out across epochs and channels to give insight.

```
number_of_bins = 18

def P_distribution(process_name, epoch, Fs, epoch_count, channel_count):
    P = []
    all_ = epoch_count * channel_count;
    current = 0;

    for i in range(epoch_count):
        P_E = []

        for j in range(channel_count):
            current += 1;
            progress_bar(process_name, current, all_)

            P_C = []
            gama_filtered = bandpass_filter(epoch[i, j], Fs, gama[0], gama[1])
            theta_filtered = bandpass_filter(epoch[i, j], Fs, theta[0], theta[1])

            gama_hilbert = hilbert(gama_filtered, axis = -1)
            gama_amplitude = np.abs(gama_hilbert)

            theta_hilbert = hilbert(theta_filtered, axis = -1)
            theta_phase = np.angle(theta_hilbert)
```

```
            assert len(theta_phase) == len(gama_amplitude)

            width = (2 * np.pi) / number_of_bins
            start = -1 * np.pi
            A = 0
            count = 0

            for _ in range(number_of_bins):
                for k in range(len(theta_phase)):
                    if theta_phase[k] >= start and theta_phase[k] < start + width:
                        count += 1
                        A += gama_amplitude[k]

                start += width

                if (count != 0):
                    A = A / count
                    P_C.append(A)

                else:
                    P_C.append(0)

                A = 0
                count = 0

            P_C = np.array(P_C)

            if P_C.sum() > 0:
                P_C = P_C / P_C.sum()

            P_E.append(P_C)

        P.append(P_E)

    return np.array(P)
```

In this code, we first iterate over the epochs and channels to be able to calculate the scalar MI value for all of the channels individually. All of the results will be stored in a dynamic array called `P` with the dimensions (epochs, channels, N).

After calculating the proper phase and amplitude for each of the individual signals, we iterate over phase bins to be able to find the amplitudes that fall within that particular phase bin.

Then we use the provided formula for calculating $P(\theta_i)$ to calculate the distribution for each of the individual signals and store them in the main array with the proper dimensions.

After that, we return the main array that contains the $P(\theta_i)$ distribution for all of the channels of all of the epochs.

6.2 MI Calculation

Now that we have successfully calculated the P distribution, we are ready to calculate the MI values.

$$MI = \frac{D_{KL}(P(\theta)||U(\theta))}{\log(n)}$$

$$D_{KL}(P(\theta)||U(\theta)) = \sum_i P[i] \log \frac{P[i]}{U[i]}$$

$$\Rightarrow MI = \frac{\sum_i P[i] \log \frac{P[i]}{U[i]}}{\log(n)}$$

```
def compute_MI(P):
    U = np.ones(number_of_bins) / number_of_bins
    MI = 0

    for i in range(number_of_bins):
        if P[i] > 0:
            MI += P[i] * np.log(P[i] / U[i])

    return (MI / np.log(number_of_bins))
```

The implemented function for MI calculation is pretty straightforward and needs no explanation.

Now we implement a vectorized MI calculation function to be able to calculate the MI value for all of the epochs and channels individually. (Being able to handle the P distribution with the dimensions (epochs, channels, N).)

```
def vectorized_MI(P, epoch_count, channel_count):
    out = []

    for i in range(epoch_count):
        out.append([])

        for j in range(channel_count):
            out[i].append(compute_MI(P[i][j]))

    return np.array(out)
```

6.3 Averaging Across Epochs and Channels

To get a single scalar MI value to get an insight of the modulation index corresponding to a patient, we average the calculated MI values across all of the epochs and channels.

```
healthy_over_epochs = healthy_MI.mean(axis = 0)
healthy_over_channels = healthy_over_epochs.mean(axis = 0)

MCI_over_epochs = MCI_MI.mean(axis = 0)
MCI_over_channels = MCI_over_epochs.mean(axis = 0)

mild_over_epochs = mild_MI.mean(axis = 0)
mild_over_channels = mild_over_epochs.mean(axis = 0)
```

This way, the scalar modulation index corresponding to each of the patients after being multiplied by a factor of 10^5 for better insight is as follows:

Healthy	114.25231641534488
MCI	98.27538813125324
Mild	104.59797252879066

Table 1: Modulation Index for all of the Patients

6.3.1 Key Observations

We clearly see that the modulation index is higher in the healthy patient in comparison with the MCI and Mild patients, which indicates a higher and more powerful modulation of the gamma band amplitude with the theta band phase.

6.4 Specific Electrode Selection

To get a better insight on modulation index values for specific electrodes (channels), we calculate the MI for these specific channels: (FP_1, FP_2, F_z, P_z).

Also note that the indexes are multiplied by a factor of 10^5 for better insight.

```
healthy_over_selected_channels = healthy_over_epochs[selected_channels,].mean() * 100000
MCI_over_selected_channels = MCI_over_epochs[selected_channels,].mean() * 100000
mild_over_selected_channels = mild_over_epochs[selected_channels,].mean() * 100000
```

Healthy	115.55066593074154
MCI	107.79242384003481
Mild	98.62853719213595

Table 2: Modulation Index for all of the Patients

6.4.1 Key Observations

In these specific electrodes, the modulation index difference between the healthy patient and the 2 other patients is much more visible.

The MI value has increased for the healthy patient and decreased for the 2 other patients, which indicated a higher and more powerful modulation in these specific channels for the healthy patient and lower for the other ones.

6.5 Phase-Amplitude Histogram

The results of this part have been provided as an insight for bonus marks related to the *Additional Insights/Visualizations* part.

To get a better insight, we use the calculated P distribution to get a histogram that corresponds to the average amplitude that falls within each phase bin with the center of that bin.

```
healthy_averaged_P = healthy_P.mean(axis = (0,1))

x_axis = []
start = -1 * np.pi
step = (2*np.pi) / number_of_bins

for _ in range(number_of_bins):
    end = start + step
    x_axis.append((start + end) / 2)
    start += step

plt.figure(figsize=(7,4))
plt.bar(x_axis, healthy_averaged_P, width=2*np.pi/number_of_bins, align="center", edgeco
plt.xticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi],
            [r"$-\pi$", r"$-\pi/2$", "0", r"$\pi/2$", r"$\pi$"])
plt.xlabel("Phase (Radians)")
plt.ylabel("Normalized Amplitude")
plt.title("Phase{Amplitude Distribution for Healthy")
plt.tight_layout()
plt.show()
```

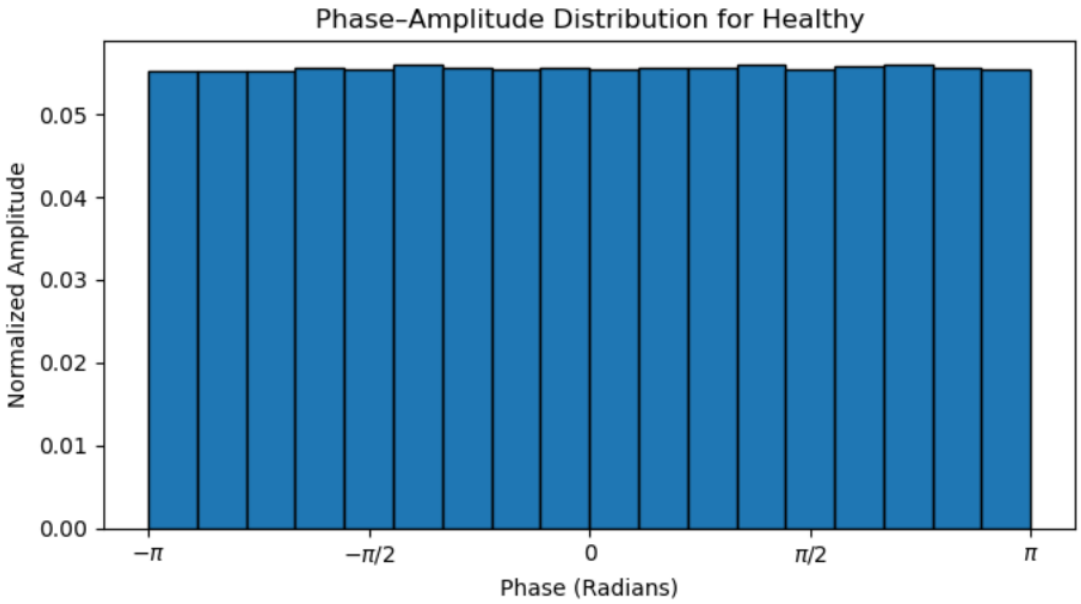

Figure 14: Phase-Amplitude Histogram for Healthy Patient

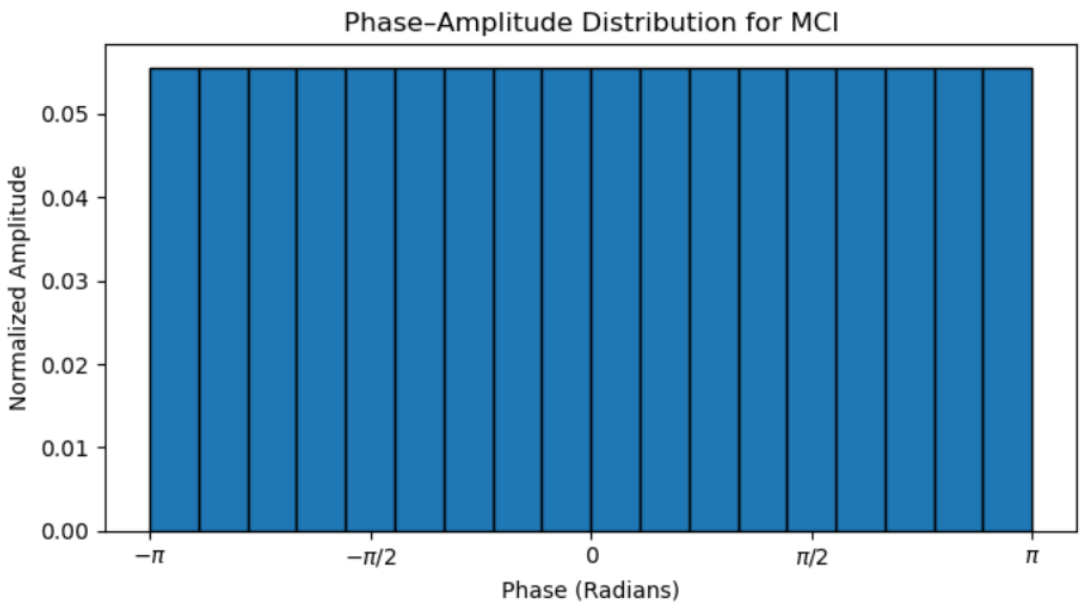

Figure 15: Phase-Amplitude Histogram for MCI Patient

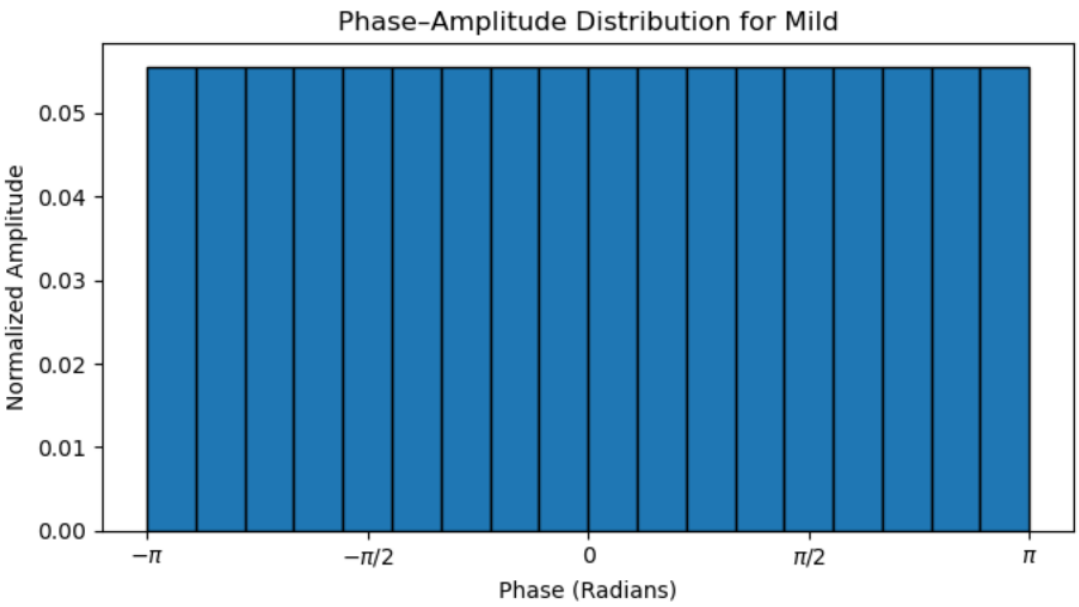

Figure 16: Phase-Amplitude Histogram for Mild Patient

6.5.1 Key Observations

Since the Phase-Amplitude Histogram for both the MCI and Mild patients is completely flat and uniform, we are sure that there is no Phase-Amplitude coupling happening in these signals.

There is a slight and small non-uniform component in the histogram corresponding to the healthy patient which indicated a higher modulation, even so slightly.

7 Comparison: MVL vs. MI

Even though the results from the 2 analysis did not match perfectly, they both gave us some insight.

The MVL analysis gave a better insight on the effect of repetition because it was applied using a sliding window method (which could have also been done for the MI method).

The MVL analysis also gave us an insight to what happens to the modulation and how it changes throughout a stimulus and when it's applied.

On the other hand, the fact that the MI analysis described the amount of modulation between the gamma band amplitude and the theta band phase with a single scalar value gave us an overall view of the situation. This single number makes it easy to compare the overall PAC strength across different patients and draw a quick conclusion.

The MI results, where the Healthy patient had a higher modulation index than the MCI and Mild patients, align with traditional clinical findings on phase-amplitude coupling in AD and MCI.

Overall, both the MVL and MI analysis helped us understand how the cognitive state of the patient effects the sigma band amplitude and theta band phase modulation, and how it changes throughout time.

A critical discrepancy exists between the two methods results. The MVL analysis produced unusually high PAC values that contradicted the expectation of reduced coupling in MCI and Mild patients. This suggests a potential bug or a flaw in the scaling of the EEG signals, making the absolute MVL values difficult to interpret clinically.

Conversely, the MI analysis yielded results consistent with the expected clinical trend, where the Healthy patient demonstrated the strongest coupling. The flat phase-amplitude histograms for the MCI and Mild groups further confirm the minimal coupling detected by the MI method.